

## Technical Report: TaskFlow API

### 1. Introduction

TaskFlow API is a data-driven, database-backed web API developed as an individual coursework project for module XJCO3011. It fully implements CRUD operations integrated with a persistent SQL database, and includes structured validation, standard HTTP status codes, and complete supporting documentation.

The system provides RESTful JSON endpoints for task management, complemented by lightweight HTML demonstration pages to support clear presentation and local testing during the oral examination. The project has been deployed live on PythonAnywhere to meet the coursework requirement for external demonstrability.

### 2. Project Aim and Motivation

The primary aim was to design, implement, and document a fully functional web API linked to a persistent SQL database, while justifying all technology and architecture decisions in line with software engineering principles.

A task management domain was selected because:

- It is a realistic, relatable use case that clearly demonstrates core API concepts.
- It supports full CRUD, data validation, status handling, and structured routing.
- It allows clear demonstration during oral assessment.
- It provides natural scope for future extension into filtering, authentication, and analytics.

This project applies modular design, separation of concerns, and disciplined version control to meet both the minimum requirements and higher-grade criteria for independent inquiry.

### 3. Technology Stack and Justification

All components were chosen to meet coursework requirements while ensuring stability, clarity, and ease of demonstration.

#### Python

Selected for its clean syntax, wide industry adoption, and strong suitability for backend API development. It supports rapid prototyping and clear code structure, ideal for coursework presentation and debugging.

#### Django

Chosen as the core web framework for:

- Stable, maintainable project structure
- Built-in database and configuration management
- Powerful ORM that minimises raw SQL
- Clear separation of models, views, and routing

#### Django REST Framework

Used to implement the JSON API layer for:

- Automated serialization and validation
- Consistent JSON response formatting
- Generic views that keep code concise and modular
- Browsable API support for demonstration

#### SQLite

Used as the database because:

- It is a file-based SQL database that satisfies coursework requirements.
- It requires no separate server and is ideal for local development.
- It supports full persistence and querying for a single-user prototype.
- It can be easily migrated to PostgreSQL in production.

#### 4. System Design and Architecture

The project follows Django's modular app architecture, with a single core application tasks managing all task-related logic.

Key components:

- `models.py`: Defines the persistent Task data structure
- `serializers.py`: Manages API data conversion and input validation
- `views.py`: Implements API endpoints and HTML demonstration views
- `urls.py`: Establishes clear, REST-style routing
- `settings.py`: Configures apps, databases, and security parameters

This separation of concerns improves maintainability, testability, and explainability during the oral examination.

#### 5. Data Model Design

A single core model Task is implemented, satisfying the requirement of one data model with full CRUD.

Fields:

- `id` (primary key)
- `title` (required, non-blank)
- `description` (optional text)
- `status` (limited choice set)
- `priority` (limited choice set)
- `due_date` (optional date field)
- `created_at` (auto-generated timestamp)
- `updated_at` (auto-updated timestamp)

Constraints on status and priority prevent invalid data. Timestamps support traceability and future analytics features.

#### 6. API Design

The API follows RESTful conventions and provides five functional endpoints, exceeding the minimum requirement of four.

Core JSON Endpoints

- GET `/api/tasks/api/` – List all tasks
- POST `/api/tasks/api/` – Create a new task
- GET `/api/tasks/<id>/` – Retrieve a single task by ID
- PUT `/api/tasks/<id>/` – Update a task by ID
- DELETE `/api/tasks/<id>/` – Delete a task by ID

Demonstration HTML Pages

- `/api/tasks/` – Task list and creation page
- `/api/tasks/select/` – Task selection interface
- `/api/tasks/page/<id>/` – Single task detail view

HTML pages support presentation without altering the core API layer. The live deployment is accessible at:

<https://yunshen.pythonanywhere.com/api/tasks/>

## 7. Validation and Error Handling

Validation is enforced at model and serializer levels to ensure data integrity.

### Validation Rules

- title is required and cannot be blank
- status and priority use predefined choices
- due\_date is optional and validated as a date

### Standard HTTP Status Codes

- 200 OK: Successful read/update
- 201 Created: Successful creation
- 204 No Content: Successful deletion
- 400 Bad Request: Invalid input
- 404 Not Found: Task ID does not exist

This follows industry best practices and supports clear debugging and demonstration.

## 8. Testing Approach

The system was validated through systematic manual testing in both local development and the live PythonAnywhere environment.

### Test cases:

- Create, read, update, and delete tasks via API and HTML
- Navigate between HTML demonstration pages
- Verify database persistence across sessions
- Test invalid inputs and non-existent IDs
- Confirm correct status codes and error messages
- Validate live deployment behaviour

All core paths are repeatable and demonstrable. Automated testing can be added in future iterations.

## 9. Challenges and Lessons Learned

### Key Challenges

- Managing Django database migrations during structural changes
- Balancing pure API functionality with clear oral demonstration workflow
- Ensuring consistent routing between API and HTML views
- Successful deployment to PythonAnywhere with correct static file and database configuration

### Lessons Learned

- Explicit routing improves maintainability and reduces errors
- Migration planning is critical in Django
- Separating API and presentation layers supports extensibility
- Documentation must be updated alongside code changes
- Deployment requires careful environment and dependency management

These reflections demonstrate reflective learning as required by the assessment.

## 10. Current Limitations

The current version is a stable demonstration prototype with intentional scope limitations:

- Single data model only

- No user authentication or authorization
- No filtering, search, or sorting endpoints
- No analytics or reporting features
- No automated test suite
- Uses SQLite rather than a production database
- Designed for assessment demonstration, not high-scale production

These limitations are realistic for a coursework prototype and provide clear direction for improvement.

## 11. Future Improvements

To advance the project toward production readiness:

1. Add user authentication and task ownership
2. Implement filtering by status, priority, and due date
3. Add analytics (overdue tasks, completion rate)
4. Upgrade from SQLite to PostgreSQL
5. Build an automated test suite with pytest
6. Generate formal PDF API documentation (Swagger)
7. Strengthen security and deployment hardening

These improvements align with higher-grade criteria for creativity and advanced functionality.

## 12. Project Links

GitHub Repository: <https://github.com/cloud12344/TaskFlow-API>

Live Deployment: <https://yunshen.pythonanywhere.com/api/tasks/>

API Documentation (PDF): [./docs/api.pdf](#)

Technical Report (source): [./docs/report.pdf](#)

## 13. Generative AI Declaration

This assessment is classified as Green Light: GenAI was used systematically and transparently.

### Tools Used

- ChatGPT
- Cursor AI assistant

### Purposes

Assisting with debugging Django routing, database migrations, view logic, and serializer validation

Helping interpret the coursework brief and ensure compliance with all technical and documentation requirements

Supporting the design of API endpoints, HTTP status code usage, and error handling logic

Refining README.md, API documentation, and technical report structure and academic wording

Supporting deployment preparation and configuration checks for PythonAnywhere

### Reflection

AI supported planning, debugging, drafting, and deployment guidance but did not replace design decisions. All suggestions were reviewed, adapted, and implemented manually. I retain full responsibility for architecture, code, documentation, and

deployment. AI accelerated iteration and helped ensure all deliverables were complete.

report.docx

XJCO3011 Course...

阅读这两个文档，根据后面这个pdf的要求，修改润色前面这个报告让报告符合标准

Technical Report: TaskFlow API

Module: XJCO3011 Web Services and Web Data

Assessment: Coursework 1 – Individual Web Services API Development Project

Word Count: Within 5-page limit

Submission Date: 21 April 2026

1. Introduction

TaskFlow API is a **data-driven web API** with full SQL database integration, developed to meet the requirements of XJCO3011 Coursework 1. This individual project implements complete **CRUD functionality**, structured routing, data validation, standard HTTP status codes, and supporting documentation.

The system provides JSON-based API endpoints for task management, paired with lightweight HTML demonstration pages to support clear presentation and local testing during the oral examination. All components follow modular software engineering principles and comply with the coursework's minimum technical standards.

2. Project Aim and Motivation

The primary aim was to **design, implement, and document a fully functional web API** linked to a persistent SQL database, while justifying all technology and architecture decisions.

现在我想把这个项目部署到PythonAnywhere上我该怎么告诉我详细步骤

可以。你这个项目是 Django + SQLite，部署到 PythonAnywhere 是比较合适的，流程也比较标准。

我给你按你当前项目来写一套 从本地到上线的完整步骤。

一、先确认你的项目适合部署

你现在项目里已经有这些基础条件：

- Django 项目结构完整
- 有 `manage.py`
- 有 `wsgi.py`
- 使用 SQLite
- API 路由已经能本地运行
- `requirements.txt` 已存在

这说明你已经具备部署基础。

二、部署前本地先做的准备

一、你最终必须完成哪些内容

1. 核心项目：一个可运行的 Web API

最低要求是：

- 至少有 1 个数据模型
- 这个数据模型必须实现完整 CRUD：
  - Create
  - Read
  - Update
  - Delete
- 连接数据库
- 至少 4 个可通过 HTTP 访问的 API endpoint
- 正确处理用户输入
- 返回 JSON（如果你有合理理由用别的格式）
- 使用正确的 HTTP 状态码和错误码
- 能够演示运行：
  - 本地运行
  - 或部署到外部平台
  - 或做成 MCP server

注意：

- SQL 数据库是推荐/默认选择

14. Conclusion

TaskFlow API fully meets the minimum technical requirements of XJCO3011 Coursework 1, including database integration, CRUD operations, validation, error handling, documentation, and live deployment. The system is modular, testable, and designed for clear oral demonstration.

While intentionally limited in scope, it provides a strong foundation for extension into authenticated, analytical, and production-ready versions. This project reflects independent development, justified design choices, and reflective learning in a GenAI-enabled workflow.

15. References

|    |        |          |                                                                               |                 |                       |
|----|--------|----------|-------------------------------------------------------------------------------|-----------------|-----------------------|
| 1. | Django | Software | Foundation.                                                                   | <i>Django</i>   | <i>Documentation.</i> |
|    |        |          | <a href="https://docs.djangoproject.com/">https://docs.djangoproject.com/</a> |                 |                       |
| 2. | Django | REST     | Framework.                                                                    | <i>Official</i> | <i>Documentation.</i> |

<https://www.django-rest-framework.org/>

3. SQLite. *SQLite Documentation*. <https://www.sqlite.org/docs.html>
4. PythonAnywhere. *Help and Documentation*. <https://help.pythonanywhere.com/>
5. TaskFlow                      API                      Live                      Deployment.  
<https://yunshen.pythonanywhere.com/api/tasks/>
6. University of Leeds. XJCO3011 Web Services and Web Data – Coursework 1 Brief, 2025–2026.